

Best Practices for GitHub Copilot

Ten battle-tested strategies to maximise code quality, reduce refactors, and get the most out of AI-assisted development – from architecture design to safe rollback workflows.



Design First

Incremental Dev

Analysis Before Code

Modular Code

Freeze Architecture Before Coding

Why Design First Matters

Jumping straight to coding without architecture means Copilot generates code that doesn't fit your system design — leading to costly refactors. Use ChatGPT or DeepSeek as a co-thinking partner to freeze architecture decisions early.

 **Clear requirements → Better code generation → Fewer rewrites → Faster time to production**

Step-by-Step: Design & Lock Architecture

1. Define API endpoints — e.g. `POST /api/retrieval` (query, filters, maxResults)
2. Design DB schema & table relationships — `documents`, `cache`
3. Choose tech stack — Node.js + TypeScript, React, PostgreSQL + Pinecone, Redis
4. Plan folder structure — `src/services`, `src/routes`, `src/models`
5. Q&A with LLM on business logic — fallbacks, stale cache handling
6. Clarify scale & constraints — 10K docs? 1M? QPS? Latency SLA?
7. Lock architecture — team alignment, write it down, then start coding

iSPOT Prompt Pattern

Instruction: Validate hybrid retrieval design before any code generation in Copilot.

Situation: RAG system for 50K technical docs. Goal: 100ms P95 latency, 95%+ recall. Uncertainty: cache at query or result level?

Persona: Architect responsible for system reliability. Want to lock design early.

Output:

- Validated API contract
- DB schema with indexing strategy
- Caching tier design (query vs. result + TTL)
- Fallback strategy (vector → BM25 → LLM fallback)

Tone: Technical, defensive, constraint-aware.

Concrete RAG Example

 Without Design First 2 weeks in, you realise there's no caching. Every query hits Pinecone → 200ms latency → SLA miss. Refactoring takes days.	 With Design First Ask ChatGPT to design hybrid retrieval with fallback and caching. Document in <code>ARCHITECTURE.md</code>. Ask Copilot to implement following the doc. Code matches design from day 1 — no mid-project refactors.
---	---

 **2-4 Hours**

Spend on design Q&A before writing any code

 **ARCHITECTURE.md**

Document all design decisions in workspace markdown

 **Lock Scale**

"10K docs today, plan for 1M growth" — share with team before coding

Incremental Development & Version Control

Small steps, frequent checkpoints, and meaningful commits keep Copilot-generated changes manageable, testable, and reversible.

Why Incremental + Git Discipline Matters

Copilot can generate large multi-file changes that are hard to test and review. Bugs accumulate and rollback becomes a nightmare. Build in small steps – 1–2 functions per request – using Git branches and frequent checkpoints.


 Easy debugging with `git bisect`, clear history, safe rollback, and collaborative code review.

Step-by-Step: Incremental Workflow

1. Create feature branch: `git checkout -b feature/hybrid-retrieval-v1`
2. Ask Copilot for small, focused task – e.g. `queryVectors()` only
3. Review generated code for logic, error handling, type safety
4. Test locally – unit tests, integration tests
5. Commit with meaningful message: `feat: add queryVectors() with 5s timeout and retry logic`
6. Next task: `queryBM25()` with same signature (incremental)
7. Repeat: Small tasks → Test → Commit → Next task

iSPOT Prompt Pattern

Instruction: Generate ONLY the `queryVectors()` function. Do NOT include `queryBM25()`, caching, or circuit breaker yet.

Situation: Hybrid retrieval service. Function must call Pinecone with 5s timeout. No retry yet. `queryBM25()` will be built separately.

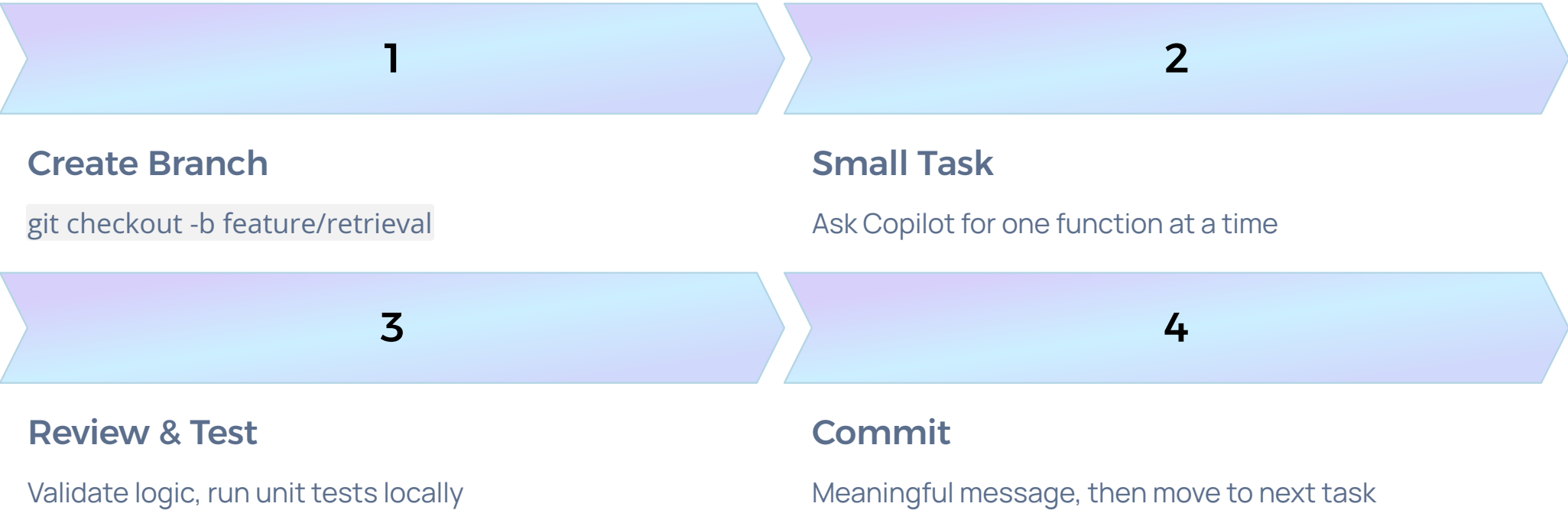
Persona: Developer building retrieval incrementally. Each step must be testable and reviewable independently.

Output:

- Function signature with TypeScript types
- Pinecone API call with proper error handling
- Unit test skeleton

Tone: Focused, minimal scope, testable.


 Never ask Copilot for an "entire feature" – always break into 1–2 function tasks.



Ask for Analysis Before Code

Intent and analysis first — then code generation. Discussing the approach before writing a single line leads to aligned solutions, fewer wrong turns, and faster convergence.

01	02	03
Describe the Problem "Our <code>queryVectors()</code> sometimes takes 10s when it should timeout after 5s. What could cause this?"	Ask for Analysis (NOT Code Yet) "Analyse the possible root causes and how to diagnose them."	Review & Discuss Approach "Does adding a <code>Promise.race()</code> timeout check work here? Any edge cases?"
04	05	
Align on Solution Get confirmation from Copilot before proceeding to implementation.	Then Ask for Code "Implement this approach in <code>queryVectors()</code> ." Review and test the result.	

iSPOT Prompt Pattern

Instruction: Analyse why `queryVectors()` to Pinecone sometimes exceeds the 5s timeout window. What are the root causes? (*First message — no code yet*)

Situation: 10–20% of requests take 10s+, exceeding SLA. Single retry on timeout, no circuit breaker yet.

Output:

- Root cause analysis (network, Pinecone overload, retry backoff?)
- Diagnostic approach (logging, metrics to capture)
- Proposed fix strategies (`Promise.race`, abort controller, etc.)

Tone: Analytical, systematic, measurement-focused.

Concrete RAG Example

Without Analysis First: Copilot adds naive query-level caching. But queries have filters that change cache behaviour. Caching returns stale results. Back to square one.

With Analysis First:

1. Message 1: "Analyse: should we cache at query level or result level?"
2. Copilot: "Cache at Pinecone response level, tag with embedding hash, TTL 1hr"
3. Discuss: "What about partial result matches?"
4. Copilot: "Use embedding similarity (0.95+) to detect cache hits"
5. Message 2: "Implement this similarity-based cache in `queryVectors()`"

✅ Result: Smart caching, fewer stale results, better performance.

Always start with "Analyse..." or "Explain..." before asking for code

Document the analysis in comments — helps future maintainers

Ask follow-up questions to validate Copilot's analysis

For complex bugs, do 2–3 rounds of analysis before code generation

Modular Code & Concise Docs

Keep functions isolated and documentation essential. Large monolithic files overwhelm Copilot's context window – small, focused modules produce dramatically better code generation.

Why Modularity + Concise Docs Matter

Large 1,000–2,000 line files are hard to understand, and Copilot gets lost in context – leading to poor code generation. Keep functions and components small (50–200 lines max) with one responsibility per file. Keep docs focused: no markdown spam, only necessary documentation.

 Faster Copilot context scanning → clearer code generation → easier maintenance.

Documentation Essentials

- ARCHITECTURE.md – high-level design, not detailed
- README.md – quick start, key concepts
- Code comments – only for "why", not "what"
- JSDoc for public APIs only
- No redundant docs (DRY principle)
- Avoid 50+ page docs, verbose comments, outdated markdown

Concrete RAG Example

 Without Modularity Copilot must understand 1,200 lines. Context overload. Generated circuit breaker doesn't match caching logic. Refactor needed.	 With Modularity Break into <code>vectors.ts</code> (100 lines), <code>bm25.ts</code> (80 lines), <code>cache.ts</code> (120 lines). Ask Copilot to "Add circuit breaker to <code>vectors.ts</code>". Copilot sees only 100 lines, generates perfect code. Tests pass immediately.
--	--

<100 Lines Per File Aim for under 100 lines; 50–80 is ideal for Copilot efficiency	1 Responsibility One responsibility per file – single responsibility principle	4 Key Doc Files ARCHITECTURE.md, API docs, key gotchas, and style guide
--	---	--

Modular Architecture: Before & After

 **BAD:** `retrievalService.ts` – 800 lines with `queryVectors`, `queryBM25`, `cache`, and `circuit breaker` all in one file.

vectors.ts

Pinecone queries only – 80 lines

bm25.ts

BM25 search only – 60 lines

cache.ts

Result caching only – 100 lines

circuit.ts

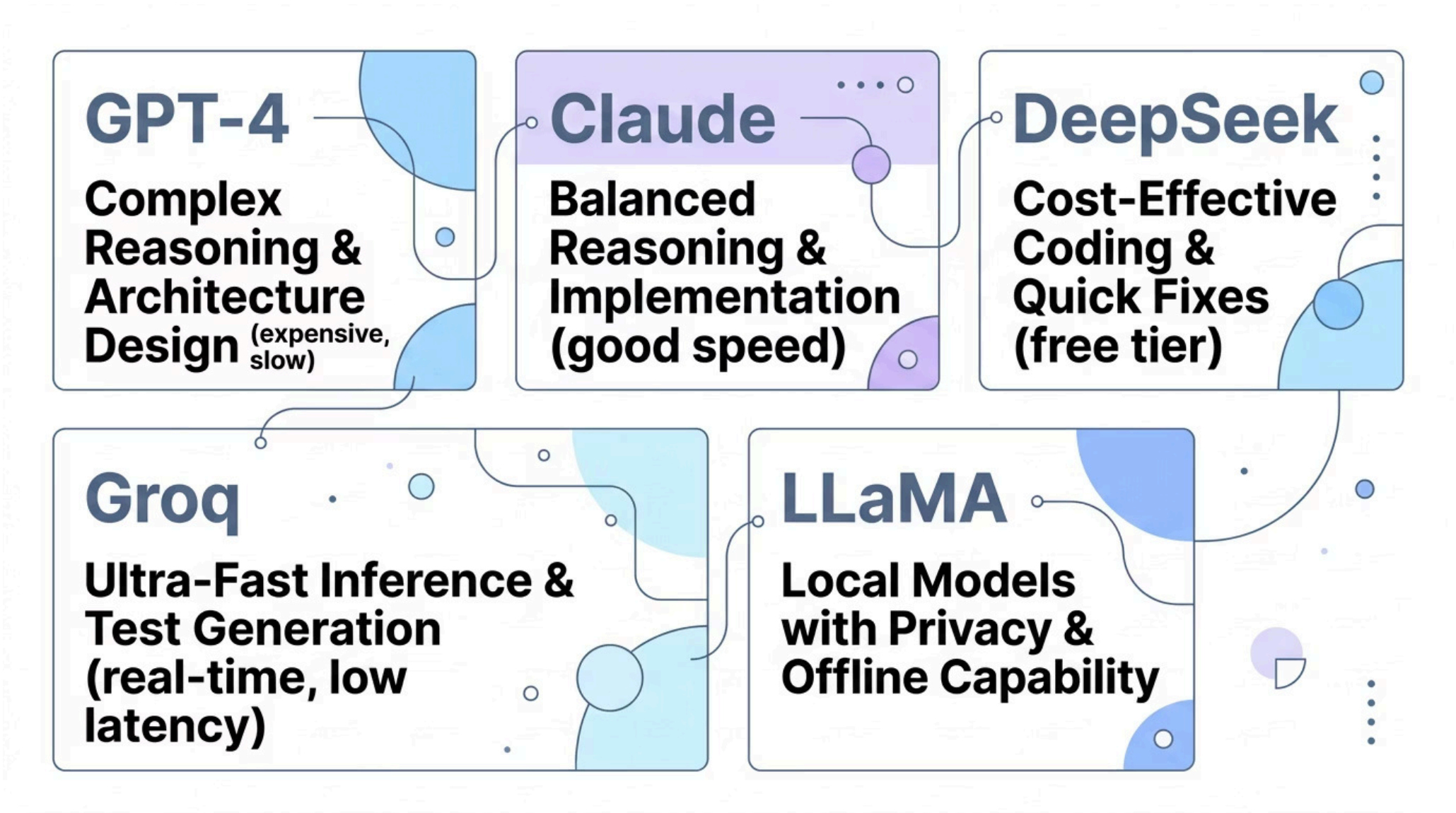
Circuit breaker only – 50 lines

retrieval.ts

Composes all modules – 70 lines

Extend VS Code with Multiple Models

Relying on a single model creates cost, latency, and capability trade-offs. Use the right model for each task — and save significantly on both time and money.



Setup: Multiple Models in VS Code

1. Install extensions: Copilot (default), Continue.dev (DeepSeek/Groq), Aider (local LLaMA)
2. Configure API keys: OpenAI, Anthropic, DeepSeek, Groq
3. Map models to tasks: Design → GPT-4, Implementation → Claude/DeepSeek, Tests → Groq, Local → LLaMA
4. Use model switcher in VS Code for each task
5. Track costs per model and benchmark latency
6. Optimise: fast models for iterative work, slow for one-time design

Cost Comparison: RAG Project

⊗ **Without Multiple Models:** GPT-4 for everything. Cost: \$300+/month just for repeated test generation. Slow iteration.

Model	Cost/Month	Use Case
GPT-4	\$50	Architecture, complex decisions
Claude	\$80	Implementation, code review
DeepSeek	Free	Coding variants, quick fixes
Groq	Free	Test generation, instant feedback

✅ **Total: \$130/month. Iteration speed: 10× faster.**

Unit Tests for Every Feature

Build tests alongside code — not after. Test-driven development catches bugs early, enables safe refactoring, and gives Copilot a way to verify its own output.

Step-by-Step: Test-Driven Workflow

1. Ask Copilot for function stub + unit test skeleton covering normal path, timeout, and null response
2. Implement function with Copilot — code should pass tests
3. Run tests locally: `npm test`
4. Add integration tests — e.g. `queryVectors()` + cache together
5. Add API tests — HTTP contract level: `POST /api/retrieval` expects 200 with `results[]`
6. Measure coverage: `npm test -- --coverage` (aim for 80%+)
7. Commit when tests pass and coverage is good

iSPOT Prompt Pattern

Instruction: Create comprehensive tests for `queryVectors()` covering:

1. Happy path: Pinecone returns results
2. Timeout: `Promise.race` expires after 5s
3. Null response: Pinecone returns empty
4. Circuit breaker open: skip Pinecone, fallback to BM25

Output: Jest unit tests, mock setup for Pinecone API, assertions for latency and error handling, integration test linking to BM25 fallback.

Tone: Defensive, comprehensive, reliability-focused.

Unit Tests

Function-level coverage — timeout, null, error paths

Integration Tests

Multi-module — `queryVectors()` + cache together

API Tests

HTTP endpoint contract — POST, response shape, status codes

✅ **With TDD:** Edge case caught in dev, never reaches production. Future refactors? Run tests, catch regressions instantly.

❌ **Without Tests:** Bug discovered by customer. Emergency fix. Loss of trust.

Pro Tips

- Write test skeletons **before** implementing functions
- Use Copilot to generate edge case tests (timeout, null, error)
- Aim for 80%+ coverage, especially error paths
- Use `Jest.mock` for external services (Pinecone, APIs)
- Run tests on commit (git hooks) to catch regressions early

Keep Workspace Context Markdown Updated

Outdated workspace docs cause Copilot to scan all files slowly and generate incorrect suggestions based on stale information. Maintain focused, current markdown and Copilot reads it first.



ARCHITECTURE.md

System design, module relationships, and data flow. Keep to 1–2 pages. Update when modules are added or restructured.



DATABASE.md

Schema, relationships, indexes, and migration notes. Update when new tables or columns are added.



API.md

Endpoints, request/response formats, authentication. Update immediately when new endpoints are added or contracts change.



CONSTRAINTS.md

Scale assumptions, latency SLAs, rate limits, and known bottlenecks. Lock these early and update as they evolve.

Concrete RAG Example

Without Context Docs: Copilot scans 50 files, takes 30s just for context loading. Generates code that conflicts with existing patterns.

With Updated Workspace Markdown:

1. Create docs/ARCHITECTURE.md (1 page, clear structure)
2. Add: "Modules: vectors, bm25, cache. Flow: vector → BM25 fallback. P95 SLA: 100ms"
3. Ask Copilot: "Based on ARCHITECTURE.md, add circuit breaker to vectors"
4. Copilot reads 1 markdown file, generates perfect circuit breaker code

✅ Context loading: 1s instead of 30s. Code quality: 10× better.

Step-by-Step: Maintain Context Docs

1. Create workspace markdown structure:
docs/ARCHITECTURE.md, docs/API.md,
docs/DATABASE.md, docs/CONSTRAINTS.md
2. Document current state — modules, endpoints, DB schema, SLAs
3. When code changes: update relevant markdown **immediately**
4. Use markdown in prompts: "Based on ARCHITECTURE.md, implement..."
5. Keep docs concise — 1–2 pages max per file
6. Link from README to all context docs
7. Version docs with code — commit markdown updates with feature commits

Logging & Training Feedback Loop

Capture what worked, improve prompt templates, and build a library of winning patterns. Without logging, great Copilot results are lost – and bad prompts get repeated.

Why Logging & Feedback Matters

You get great results from Copilot but don't document them → forget how you did it → repeat bad prompts → waste time. Log successful and failed interactions. Build a prompt template library.

Step-by-Step: Build Feedback Loop

1. Create prompt library: docs/prompts/ with files for design, implementation, tests, debugging
2. Log every significant Copilot interaction – prompt text, model, result quality, time, cost, team feedback
3. Rate results: Good (reuse), Bad (improve), Excellent (extract pattern)
4. Extract patterns from excellent results and template them
5. Share templates with team via docs/prompts/LIBRARY.md
6. Measure improvement: track time-to-completion, quality, iterations
7. Iterate: refine prompts based on feedback, retest

Sample Log Entry

Date	2025-11-20
Prompt	"Design hybrid retrieval with iSPOT pattern"
Model	GPT-4
Result Quality	EXCELLENT
Time Spent	15 min (vs 60 min manual)
Iterations	1 (first result was perfect)
Team Feedback	"Perfect architecture, no changes needed"
Cost	\$0.50
Template	design-hybrid-retrieval
Reuse Count	3 (saved 45 min each)



Log Everything

Every significant Copilot interaction – design, code, tests



Rate Results

Excellent / Good / Bad / Needs Rework – extract templates only from Excellent



Share Templates

Team wiki or GitHub – consistency improves across all projects



Track Metrics

Time saved, iterations reduced, quality improvement over time



With Feedback Loop: Use template, get excellent design immediately – 15 min vs 60 min. Team learns the pattern, reuses template, consistency improves. Capture winning patterns. Reduce design time 4×.

Housekeeping Rules

Why Housekeeping Matters

A messy project with dead code, inconsistent naming, and cluttered folders confuses Copilot — slowing scanning and degrading code generation quality. Regular cleanup keeps the codebase sharp.

1

Dead Code

Remove unused functions, tests, and modules using linter

2

Naming

Enforce consistent camelCase, PascalCase, UPPER_SNAKE_CASE

3

Folders

Keep organised — no dump folders, clear src/ structure

4

Dependencies

Remove unused npm packages with npm prune

iSPOT Prompt Pattern

Instruction: Identify and remove all unused functions in retrieval service. Then reorganise folder structure per style guide.

Situation: 1,200 lines, 5 unused functions (validateQuery, etc.). Folder src/retrieval/ has misc .js files. Goal: clean, organised, naming consistent.

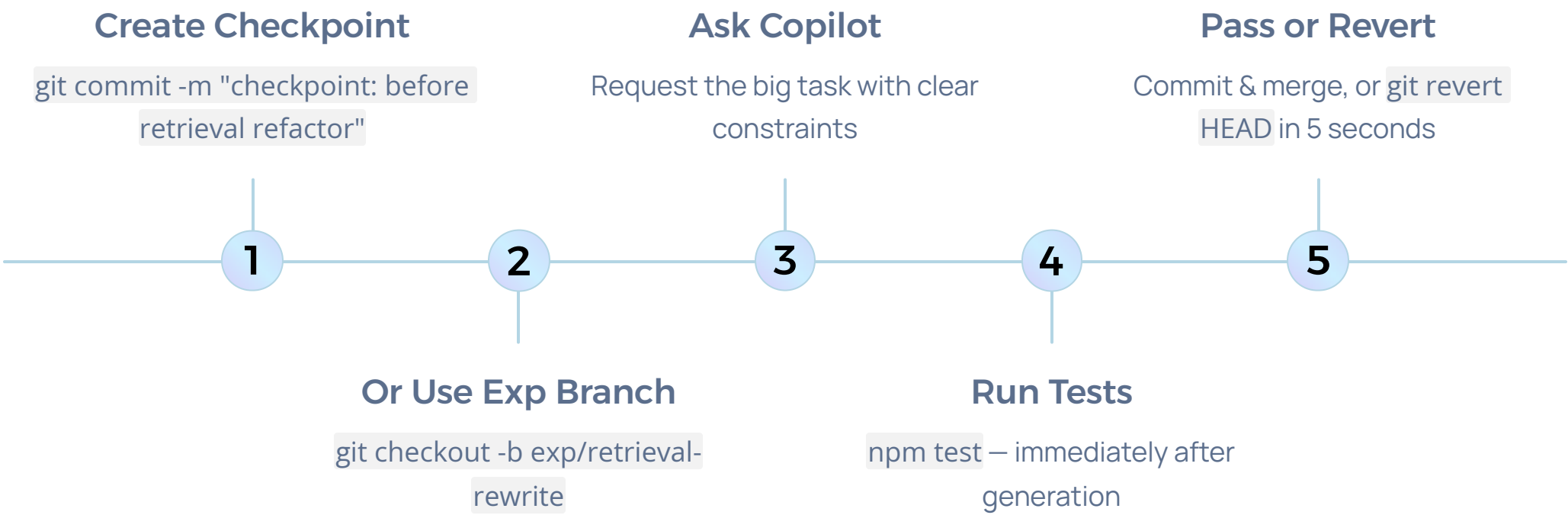
Output:

- List of unused functions to delete
- Refactored folder structure
- Updated imports to match new locations
- Linting report

✔ **With Housekeeping:** Run ESLint, remove 8 unused functions (1,200 → 1,000 lines). Reorganise into src/vectors/, src/bm25/, src/cache/. Copilot sees clean codebase → generates perfect code. Team velocity up 20%.

Restore Points & Checkpoints

Create checkpoints before asking Copilot for big changes. Safe experimentation and quick rollback give your team confidence in AI-assisted development.



Concrete RAG Example

Without Checkpoints: Spent 2 hours on refactor. Tests fail. Manually trying to revert changes. 3 hours lost. Team frustrated.

With Checkpoints:

1. Before asking Copilot: git commit -m "checkpoint: before refactor"
2. Ask Copilot for refactor — tests fail immediately
3. git revert HEAD → back to checkpoint in 5 seconds
4. Ask again with clearer constraints: "Include proper error handling in circuit breaker"
5. Second attempt passes tests. Done in 30 min total.

✔ Safe experimentation. Quick rollback. Fail fast, learn fast.

Restore Strategies

→ Copilot Restore

Native undo in Copilot — last N actions, instant

→ Git Commits

Checkpoint before major refactor — git revert for complete rollback

→ Git Branches

Experimental work on separate branch — merge only when confident

→ Git Stash

Save work-in-progress without committing — git stash pop to restore